APRENDIZAGEM POR REFORÇO

Luís Morgado

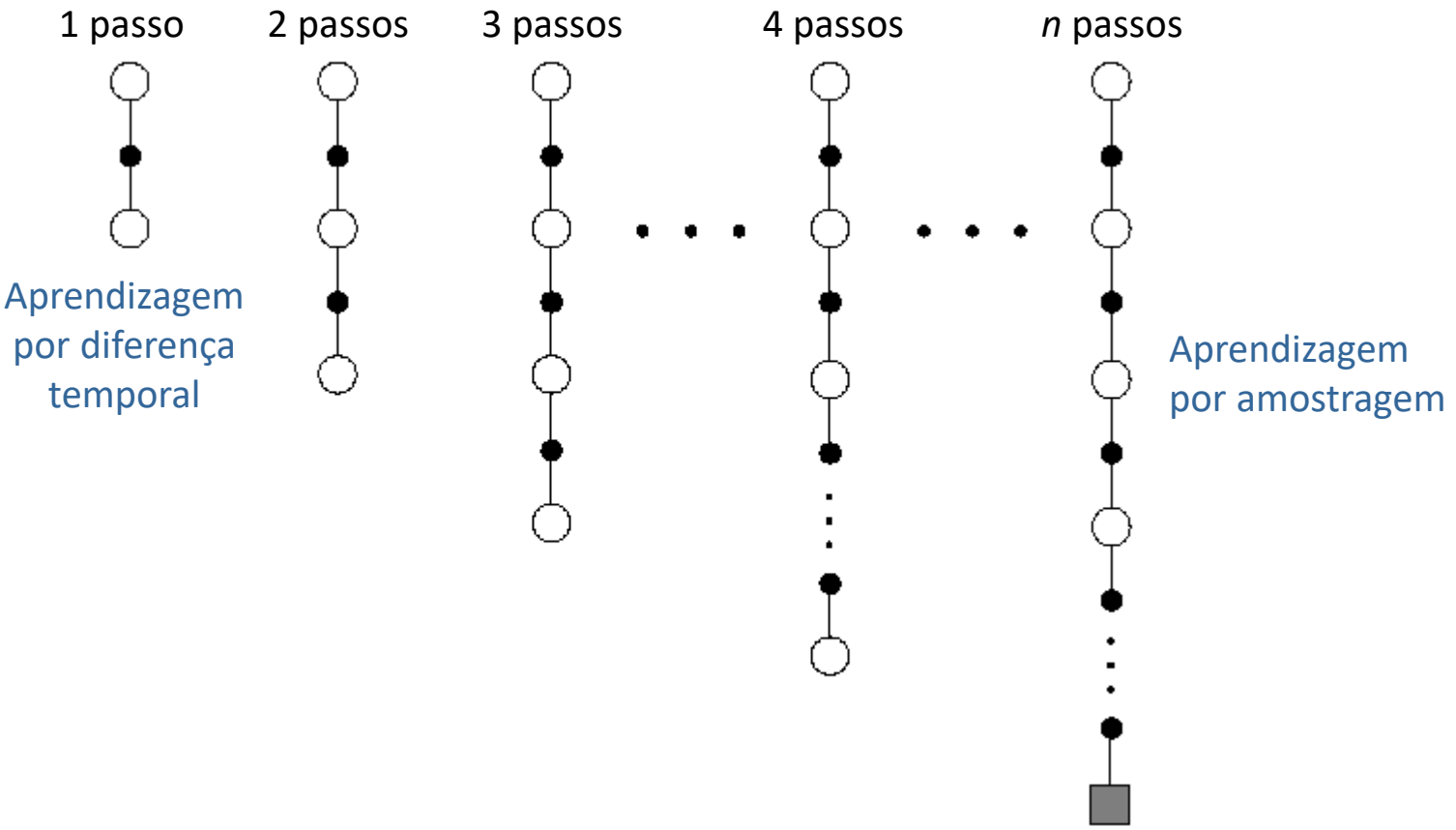
ISEL-DEI

APRENDIZAGEM POR REFORÇO

- **PROBLEMAS**
  - Complexidade dos espaços de estados
  - Tempo de convergência
- **SOLUÇÕES**
  - Utilização de modelos do mundo
  - Memória de experiência
  - Generalização / Abstracção
    - Redes neuronais
    - Modelos hierárquicos
  - Arquitecturas híbridas

FORMAS DE ACELERAR A APRENDIZAGEM

Recordar resultado de acções realizadas

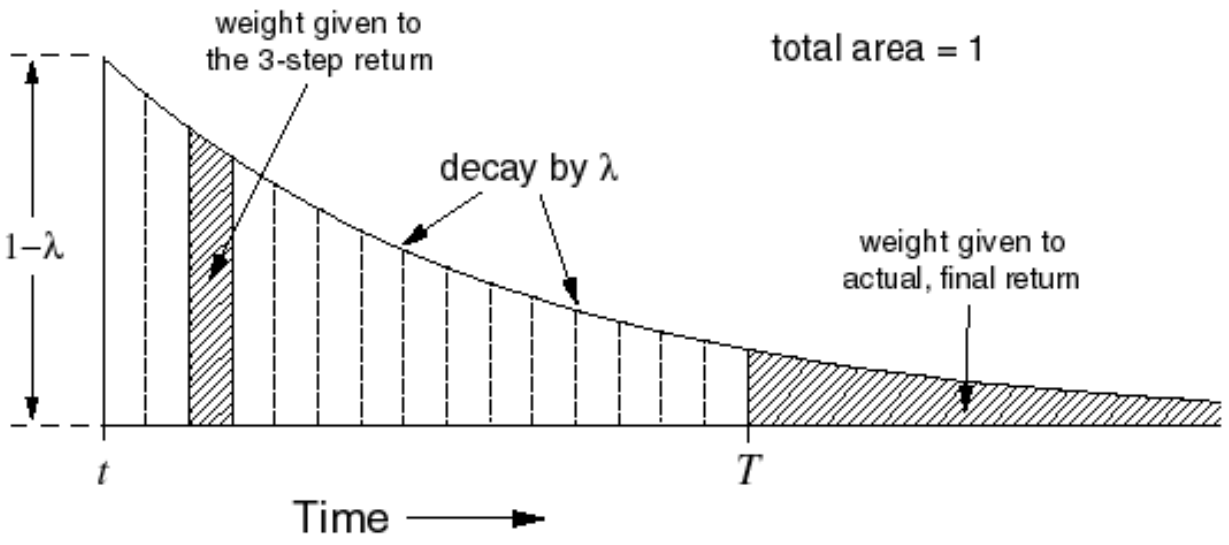


Adaptado de [Sutton & Barto, 1998]

FORMAS DE ACELERAR A APRENDIZAGEM

Ponderação da influência da recordação em função da distância no tempo

Ponderação da recordação



$\lambda \in [0,1]$ – Factor de ponderação de decaimento

Adaptado de [Sutton & Barto, 1998]

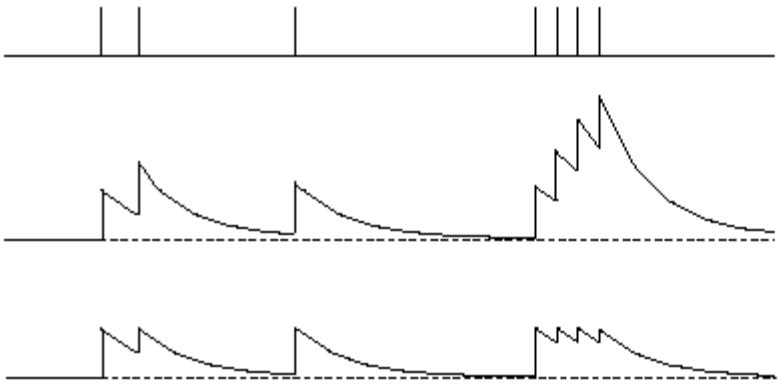
PROPAGAÇÃO DE INFLUÊNCIA

- **Rasto de eligibilidade (*Eligibility Traces*):** $e_t(s) \in \Re^+$
  - **Rasto de acumulação**
    - Acumula influência das recordações

$$e_t(s) = \begin{cases} \gamma\lambda e_{t-1}(s) & \text{if } s \neq s_t \\ \gamma\lambda e_{t-1}(s) + 1 & \text{if } s = s_t \end{cases}$$

  - **Rasto de substituição**
    - Substitui influência das recordações

$$e_t(s) = \begin{cases} \gamma\lambda e_{t-1}(s) & \text{if } s \neq s_t \\ 1 & \text{if } s = s_t \end{cases}$$



Tempo de ocorrência do estado

Rasto de acumulação

Rasto de substituição

Adaptado de [Sutton & Barto, 1998]

ALGORITMO *SARSA*(λ)

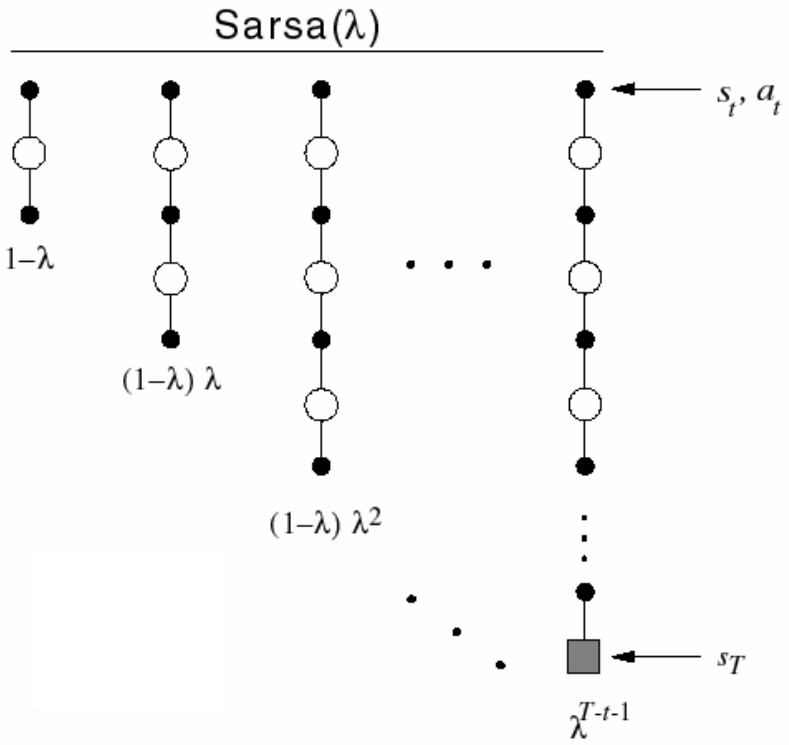
Pondera a influência da recordação em função da distância no tempo ao longo de vários passos

- Mantém registo de rasto de eligibilidade para cada par (*s*, *a*)

$e(s,a)$ – **Rasto de eligibilidade**

$$\delta_t = r_{t+1} + \gamma Q_t(s_{t+1}, a_{t+1}) - Q_t(s_t, a_t)$$

$$Q_{t+1}(s,a) = Q_t(s,a) + \alpha \delta_t e_t(s,a)$$



[Sutton & Barto, 1998]

ALGORITMO *SARSA*(λ)

Initialize $Q(s,a)$ arbitrarily

Repeat (for each episode) :

$e(s,a) = 0,$ for all s,a

Initialize s,a

Repeat (for each step of episode) :

Take action $a,$ observe r,s'

Choose a' from s' using policy derived from Q (e.g. ε - greedy)

$\delta \leftarrow r + \gamma Q(s',a') - Q(s,a)$

$e(s,a) \leftarrow e(s,a) + 1$

For all s,a :

$Q(s,a) \leftarrow Q(s,a) + \alpha\delta e(s,a)$

$e(s,a) \leftarrow \gamma\lambda e(s,a)$

$s \leftarrow s'; a \leftarrow a'$

Until s is terminal

[Sutton & Barto, 1998]

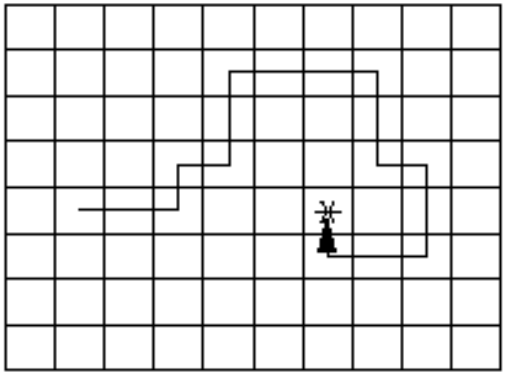
ALGORITMO *SARSA*(λ)

Os rastos de elegibilidade podem acelerar a aprendizagem

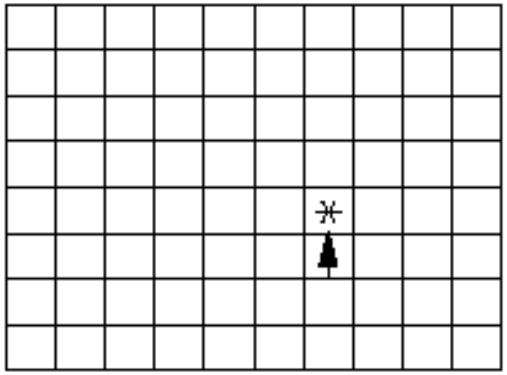
- Mais informação disponível
- Pode produzir as políticas deficientes
  - Reprodução de erros anteriores

Exemplo:

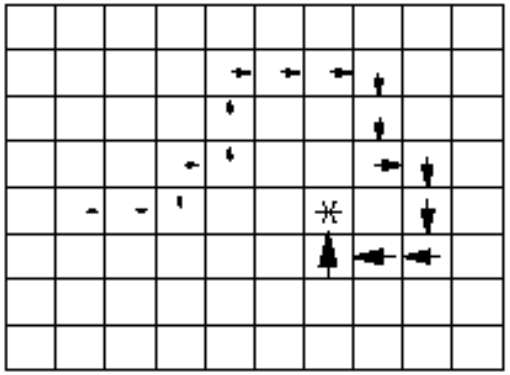
Percurso realizado



Valores de acção actualizados por 1 passo



Valores de acção com algoritmo *SARSA*(λ) com λ = 0.9



Adaptado de [Sutton & Barto, 1998]

APRENDIZAGEM COM MODELOS

Model-Based Reinforcement Learning

- Aprendizagem de um modelo do mundo
  - Representação interna das características relevantes do domínio do problema
    - **Estrutura** (estado)
    - **Dinâmica** (transição de estado)
    - **Valor** (recompensa)
- **Modelo do mundo**
  - Utilizado para simular experiência
  - Suporta planeamento
- Necessidade de manter o modelo do mundo actualizado e consistente com a experiência

MODELO DO MUNDO

Representação interna do domínio do problema

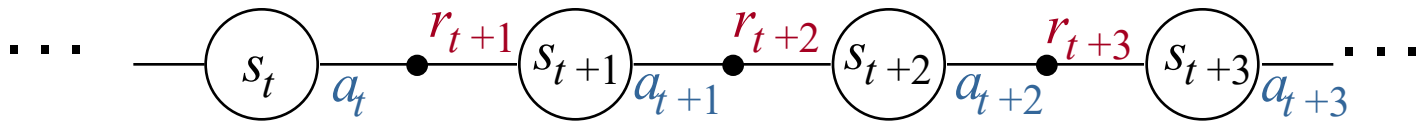
S – conjunto de estados do mundo

$A(s)$ – conjunto de acções possíveis no estado $s \in S$

$T(s,a,s')$ – probabilidade de transição de s para s' através de a

$R(s,a,s')$ – retorno esperado na transição de s para s' através de a

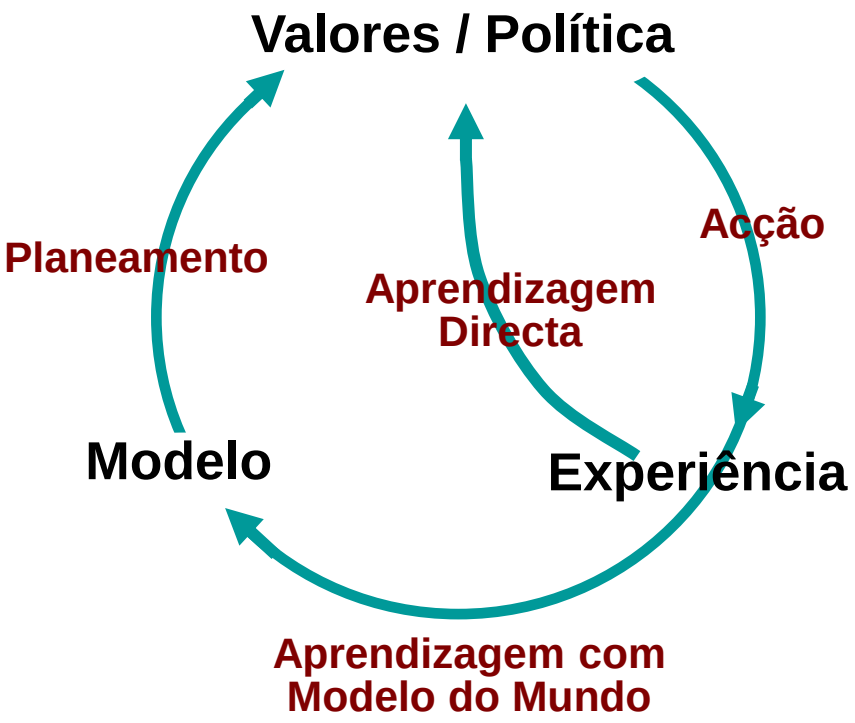
Aprendizagem: modelo do mundo actualizado a partir da experiência



APRENDIZAGEM E PLANEAMENTO

Com base num modelo do mundo é possível complementar a aprendizagem directa a partir da experiência, com aprendizagem através de planeamento interno

- O planeamento é baseado na simulação do que pode acontecer com base na experiência do que aconteceu no passado



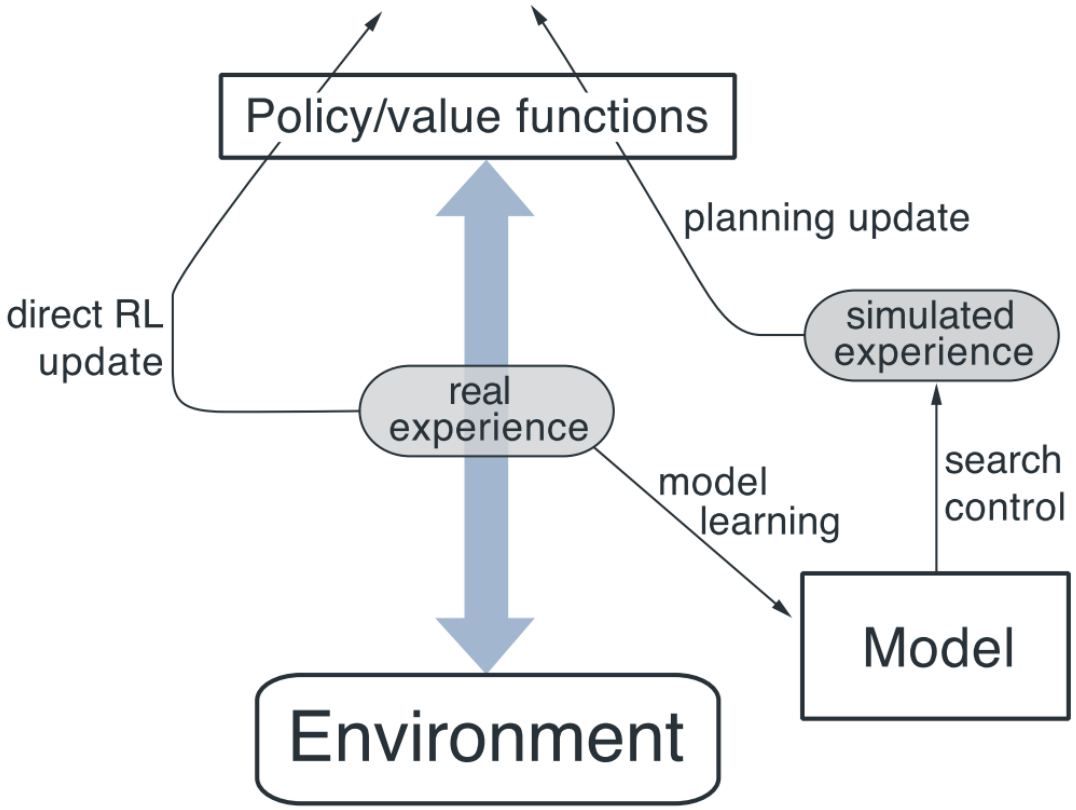
Adaptado de [Sutton & Barto, 1998]

APRENDIZAGEM E PLANEAMENTO

- **Experiência**
  - Aprendizagem de valores e políticas de acção
  - Aprendizagem de modelos do ambiente
- **Planeamento**
  - Simulação da experiência
  - Aprendizagem indirecta a partir de modelos internos
  - Iteração de comportamentos
    - Não possível directamente no ambiente

ARQUITECTURA DYNA

Arquitectura para aprendizagem por reforço que utiliza a aprendizagem directa a partir da experiência, complementada com a aprendizagem por simulação com base num modelo interno do mundo



[Sutton & Barto, 2020]

ALGORITMO DYNA-Q

Ambientes deterministas

$$Model(s,a) = \begin{cases} S \\ A \\ \hat{T}(s,a) \\ \hat{R}(s,a) \end{cases}$$

$$T: S \times A \rightarrow S$$
$$R: S \times A \rightarrow \mathbb{R}$$

$$\hat{T}(s,a) = s'$$
$$\hat{R}(s,a) = r$$

Ambientes não deterministas

$$Model(s,a) = \begin{cases} S \\ A \\ \hat{T}(s,a,s') \\ \hat{R}(s,a) \end{cases}$$

$$T: S \times A \times S \rightarrow [0,1]$$

$$\hat{T}(s,a,s') = \frac{\text{Number of (s, a, s') experiences}}{\text{Number of (s, a) experiences}}$$

$$\hat{R}(s,a) = \frac{\text{Sum of rewards received by taking action a in state s}}{\text{Number of (s, a) experiences}}$$

[Sutton & Barto, 2020]

ALGORITMO DYNA-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Do forever:

(a) $s \leftarrow$ current (nonterminal) state

(b) $a \leftarrow \varepsilon$-greedy(s,Q)

(c) Execute action a; observe resultant state, s', and reward, r

(d) $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

(e) $Model(s,a) \leftarrow s', r$ (assuming deterministic environment)

(f) Repeat N times:

$s \leftarrow$ random previously observed state

$a \leftarrow$ random action previously taken in s

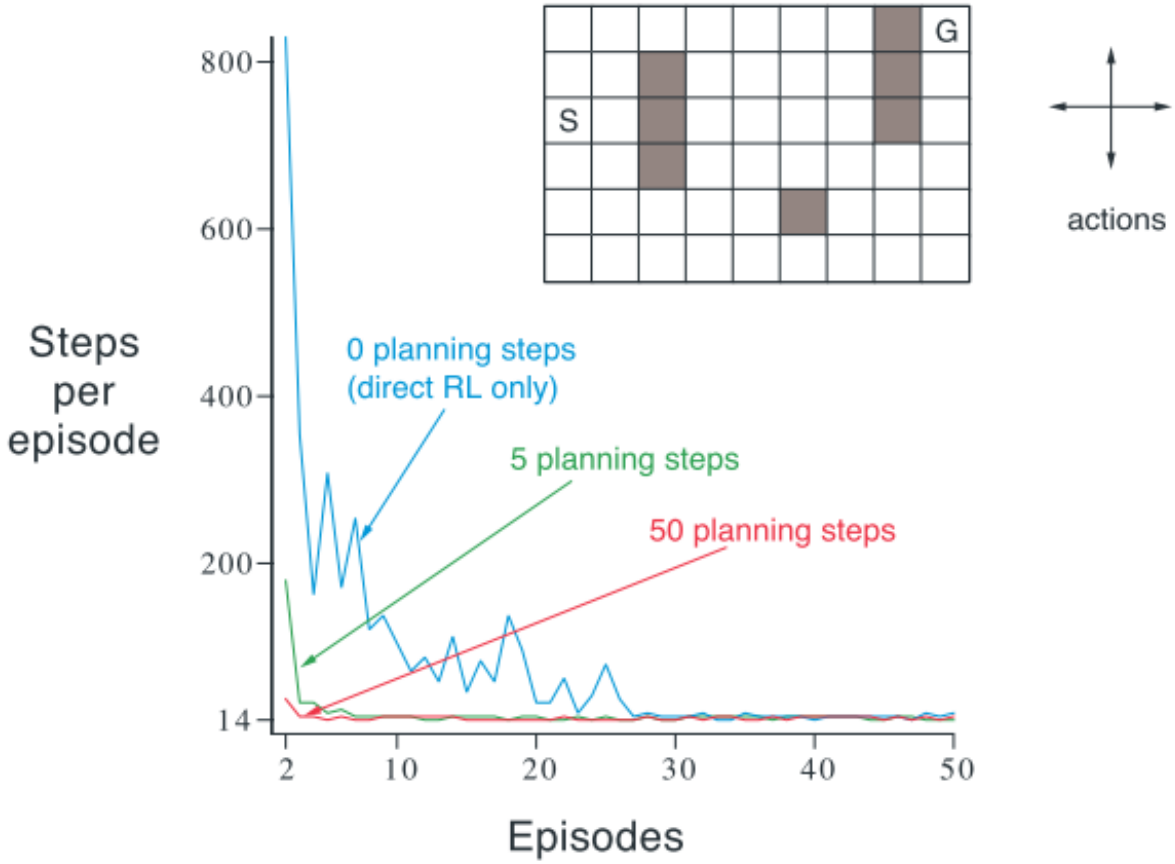
$s', r \leftarrow Model(s,a)$

$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

[Sutton & Barto, 2020]

ALGORITMO DYNA-Q

Exemplo:

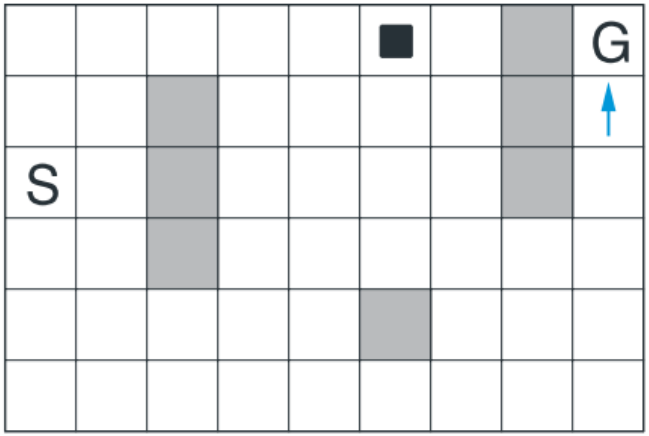


[Sutton & Barto, 2020]

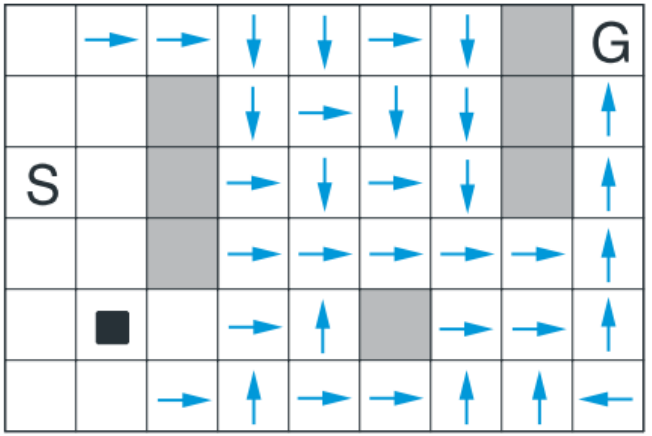
ALGORITMO DYNA-Q

Exemplo:

Sem planeamento (n = 0)



Com planeamento (n = 50)



n – Número de simulações internas por passo de execução

Adaptado de [Sutton & Barto, 2020]

ALGORITMO DYNA-Q COM PRIORIDADE

Varrimento com prioridade (*Prioritized Sweeping*)

- **Objectivo: Aumentar a eficiência do planeamento**
- Simulações e actualizações de valor de pares estado-acção específicos de acordo com uma medida de prioridade
  - Maior prioridade de pares estado-acção com grande variação de valor (mais informação – redução de incerteza)
  - Actualização de valor retrospectiva
    - Maior prioridade de pares estado-acção que levam a outros pares estado-acção com grande variação de valor
      - Varrimento de encadeamentos de transições

ALGORITMO DYNA-Q COM PRIORIDADE

Varrimento com prioridade (*Prioritized Sweeping*)

Initialize $Q(s,a)$, $Model(s,a)$, for all s, a, and $PQueue$ to empty
Do forever:

(a) $S \leftarrow$ current (nonterminal) state

(b) $A \leftarrow policy(S,Q)$

(c) Execute action A; observe resultant reward, R, and state, S'

(d) $Model(S,A) \leftarrow R, S'$

(e) $P \leftarrow |R + \gamma \max_a Q(S',a) - Q(S,A)|$.

(f) if $P > \theta$, then insert S, A into $PQueue$ with priority P

(g) Repeat n times, while $PQueue$ is not empty:

$S, A \leftarrow first(PQueue)$

$R, S' \leftarrow Model(S,A)$

$Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

Repeat, for all $\bar{S}, \bar{A}$ predicted to lead to S:

$\bar{R} \leftarrow$ predicted reward for $\bar{S}, \bar{A}, S$

$P \leftarrow |\bar{R} + \gamma \max_a Q(S,a) - Q(\bar{S},\bar{A})|$.

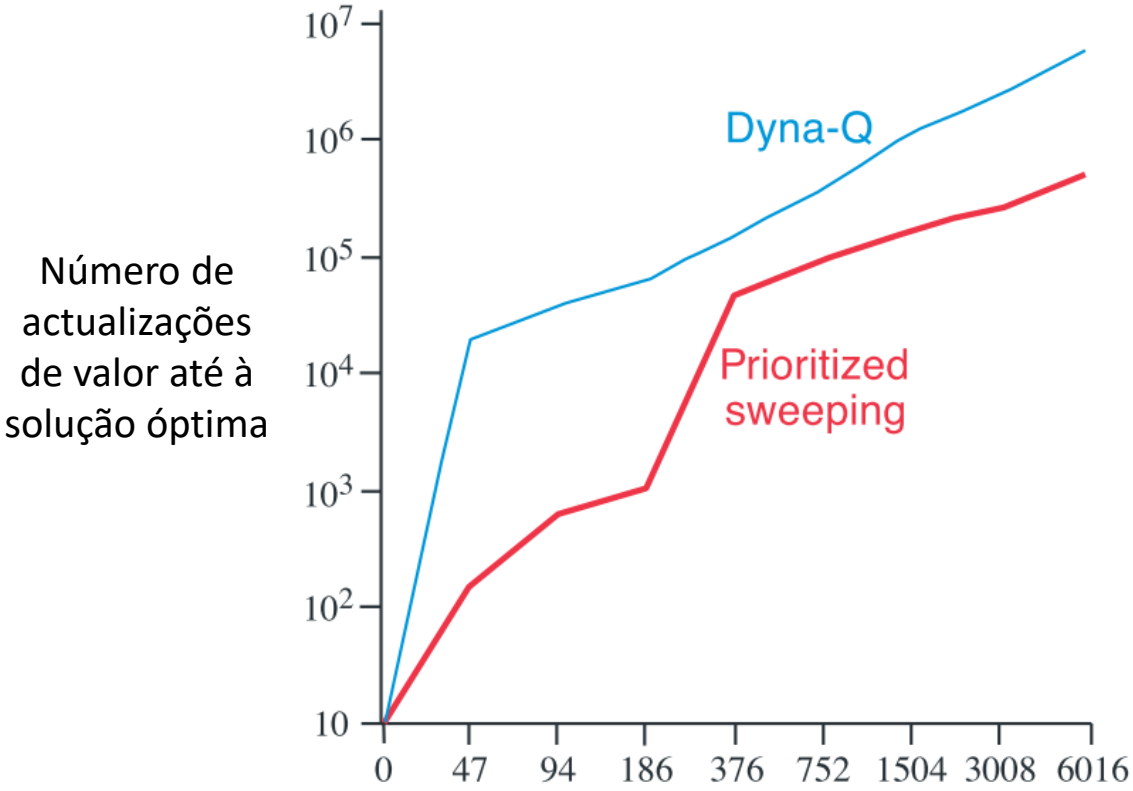
if $P > \theta$ then insert $\bar{S}, \bar{A}$ into $PQueue$ with priority P

[Sutton & Barto, 2020]

ALGORITMO DYNA-Q COM PRIORIDADE

Varrimento com prioridade (*Prioritized Sweeping*)

Exemplo:



Adaptado de [Sutton & Barto, 2020]

APRENDIZAGEM POR REFORÇO *DYNA-Q*

Mecanismo de aprendizagem

```
1.  class DynaQ extends QLearning:
2.      iniciar(mem_aprend, sel_accao, alfa, gama, num_sim):
3.          super.iniciar(mem_aprend, sel_accao, alfa, gama):
4.          this.num_sim = num_sim
5.          modelo = new ModeloTR()

6.      aprender(s, a, r, s'):
7.          super.aprender(s, a, r, s')          Actualizar Q(s,a)
8.          modelo.actualizar(s, a, r, s')       Actualizar modelo
9.          simular()                            Realizar simulação interna

10.     simular():
11.         repeat num_sim:                      Simular passos de aprendizagem com base no modelo
12.             s, a, r, s' = modelo.amostrar()
13.             super.aprender(s, a, r, s')
```

APRENDIZAGEM POR REFORÇO *DYNA-Q*

Modelo de transição e recompensa

```
1.  class ModeloTR:
2.      iniciar():
3.          T = {}
4.          R = {}

5.  actualizar(s, a, r, s'):
6.      T[(s, a)] = s'        Actualizar modelo de transição (determinista)
7.      R[(s, a)] = r         Actualizar modelo de recompensa (determinista)

8.  amostrar():
9.      s, a = choice(T.keys())
10.     s' = T[(s, a)]
11.     r = R[(s, a)]
12.     return s, a, r, s'
```

REFERÊNCIAS

[Russel & Norvig, 2003]

S. Russell, P. Norvig, “Artificial Intelligence: A Modern Approach”, 2nd Edition, Prentice Hall, 2003

[Sutton & Barto, 1998]

R. Sutton, A. Barto, “Reinforcement Learning: An Introduction”, MIT Press, 1998

[Poole & Mackworth, 2010]

D. Poole, A. Mackworth, “Artificial Intelligence: Foundations of Computational Agents”, Cambridge University Press, 2010

[Raghavan *et al.*, 2019]

A. Raghavan, J. Hostetler, S. Chai, “Generative Memory for Lifelong Reinforcement Learning”, 2019

[Cai *et al.*, 2020]

S. Cai, S. Bileschi, E. Nielsen, F. Chollet, “Deep Learning with JavaScript: Neural networks in TensorFlow.js”, Manning Publications, 2020

[Zheng *et al.*, 2018]

G. Zheng, F. Zhang, Z. Zheng, Y. Xiang, N. Yuan, X. Xie, Z. Li, “DRN: A Deep Reinforcement Learning Framework for News Recommendation”, WWW '18: Proceedings of the World Wide Web Conference, 2018